

Especificação técnica da Exportação de dados do Espaider

Identificador: *BI_SOLICITACOES_SUPORTEESPASIDER*

Versão 4.0

1. Identificação

Este documento descreve a forma de utilização da funcionalidade de Exportação de dados do sistema *Espaider*®, os métodos disponíveis, como devem ser acessados, os parâmetros de filtros e parâmetros de retorno configurados.

2. Histórico de Revisão

Data	Versão	Descrição	Autor
09/01/2026	1.0	Versão Inicial	Gabriel Andreas Cristofolini
17/01/2026	2.0	Versão atualizada	Gabriel Andreas Cristofolini
20/01/2026	3.0	Versão atualizada	Gabriel Andreas Cristofolini
07/02/2026	4.0	Versão atualizada	Gabriel Andreas Cristofolini

3. Descrição da funcionalidade

A API utiliza a arquitetura *REST (REpresentational State Transfer)* para chamada do serviço, com parâmetros de filtros e retorno em *JSON*.

Serão solicitados parâmetros de entrada (token, identificador e filtros) que serão utilizados respectivamente para autenticação, identificação da configuração de exportação e seleção dos dados a serem consultados/exportados do Espaider. Após a autenticação do token, identificação da configuração e processamento dos filtros, serão retornados os dados no formato JSON.

Os campos que possibilitam ser filtrados já estão pré-configurados na configuração de exportação no sistema Espaider. Todos os filtros adicionais informados na chamada do método, que não estejam previamente configurados na Exportação, serão desprezados.

Identificador da configuração: BI_SOLICITACOES_SUPORTEESPAIDER

Token: O token deve ser solicitado ao consultor da Fácil responsável pelo projeto no momento de implementação das rotinas, ou à Equipe de Apoio ao Cliente da Fácil.

4. Endereços da API

<http://localhost/Arauz/WCF/WCFExportaDados/WCFExportaDados.svc>

Definições dos métodos:

<http://localhost/Arauz/WCF/WCFExportaDados/WCFExportaDados.svc/help>

5. Especificação para desenvolvimento

A URL de chamada da API deve ser preenchida com o Token e o código Identificador, conforme modelo abaixo:

Url =

"ExportaDados?Token={5i/1RKSJupbiJQy0NQqUPDstPHQ8K5thKcz5m7KZQRg=}&Identificador={PesquisaProcesso}"

O "tipo" de método na chamada da API deverá ser "Method.POST".

ATENÇÃO: O token deve ser solicitado ao consultor da Fácil responsável pelo projeto no momento de implementação das rotinas, ou à Equipe de Apoio ao Cliente da Fácil.

Exemplo em c#:

```
//Endereço do sistema para acesso à API
RestClient client = new RestClient("http://localhost/Esp5.01/WCF/WCFExportaDados/WCFExportaDados.svc");

//API a ser acessar
RestRequest request = new RestRequest("ExportaDados?Token={Token}&Identificador={Identificador}", Method.POST);
//Preenchimento dos parâmetros de url
//Preenchimento do TOKEN de acesso
request.AddParameter("Token", "5i/1RKSJupbiJQy0NQqUPDstPHQ8K5thKcz5m7KZQRg=", ParameterType.UrlSegment);
//Preenchimento do identificador de configuração de exportação
request.AddParameter("Identificador", "CadastroProcessos", ParameterType.UrlSegment);
```

Figura 1) Exemplo de chamada da API.

6. Estrutura de filtros

A API solicita parâmetros de filtros que devem ser informados dentro de um objeto denominado [BlocoFiltros]. Segue abaixo a definição da estrutura de filtros esperada pela API:

Lista do tipo [BlocoFiltros]			
Campo	Tipo	Preenchimento	Observação
Identificador	Alfanumérico	Obrigatório	Identificador do bloco de filtros cadastrado
Filtro	List<Filtro>	Obrigatório	Lista de filtros do tipo [Filtro]

Lista do tipo [Filtro]			
Campo	Tipo	Preenchimento	Observação
Identificador	Alfanumérico	Obrigatório	Nome do campo configurado no Espaider
Valor	- -	Obrigatório	Valor que deve ser utilizado no filtro. IMPORTANTE: O Tipo/formato dependerá do campo ao qual o filtro se refere. Esta informação estará especificada mais adiante no respectivo campo do filtro.

Exemplo em c#:

```
List<Filtros> listaFiltros = new List<Filtros>();

//Bloco inicial de filtro
Filtros BlocoFiltro = new Filtros();
BlocoFiltro.Identificador = "ProcessoTrabalhista";

//Lista de filtros
BlocoFiltro.Filtro = new List<Filtro>();

//Campo de filtro
Filtro campoFiltro = new Filtro();
campoFiltro.Identificador = "NomeCliente";
campoFiltro.Valor = "Fácil";
//Adicionando filtro criado a lista de filtros do bloco
BlocoFiltro.Filtro.Add(campoFiltro);

//Campo de filtro
campoFiltro = new Filtro();
campoFiltro.Identificador = "NomeCategoria";
campoFiltro.Valor = "Trabalhista";
//Adicionando filtro criado a lista de filtros do bloco
BlocoFiltro.Filtro.Add(campoFiltro);

//Preenchimento da lista de filtros no corpo do "POST"
request.AddJsonBody(new List<Filtros> { BlocoFiltro });
```

Após preencher as informações de filtros, deve-se realizar a chamada.

Exemplo em c#:

```
- //Execução da chamada da API
- IRestResponse<InformacoesRetorno> retorno = client.Execute<InformacoesRetorno>(request);
//Pegando o retorno já formatado
var ObjetoRetorno = retorno.Data;
```

As classes utilizadas nos exemplos foram criadas da seguinte forma (*exemplo em c#*):

```
public class BlocoFiltros
{
    public string Identificador { get; set; }
    public List<Filtro> Filtro { get; set; }
}

public class Filtro
{
    public string Identificador { get; set; }
    public string Valor { get; set; }
}
```

7. Filtros

Segue abaixo especificação técnica dos filtros configurados para a exportação de dados:

Bloco de Filtros				
Identificador do bloco de filtro		Preenchimento	Operador	
DATAS		Opcional	E	
Filtros				
Identificador	Preenchimento	Tipo	Operador	Observações
DATAMOVIMENTACAO_ATE	Opcional	Data e hora	MenorOuIgual	Data no formato dd/MM/yyyy HH:mm:ss.
DATAMOVIMENTACAO_DE	Opcional	Data e hora	MaiorOuIgual	Data no formato dd/MM/yyyy HH:mm:ss.

8. Estrutura de retorno

Após validar e processar os parâmetros enviados, o Espaider realizará um retorno de forma síncrona com os dados encontrados. O retorno possuirá a seguinte estrutura de dados:

Objeto [InformacoesRetorno]		
Campo	Tipo	Observação
ListaRegistros	List<RegistrosRetorno>	Esta lista possui os registros retornados pela busca no Espaider em um objeto do tipo [RegistrosRetorno].
Situacao	Char	Será preenchido com “S” para sucesso, ou “E” quando ocorrer algum erro no processamento da chamada.
MensagemRetorno	Alfanumérico	Será preenchido somente quando ocorrer algum erro na execução/processamento da chamada.
URLPaginacao	URL	Será preenchido com a URL da próxima página dos registros encontrados. Este campo será retornado vazio quando não houver uma próxima página.
ListaURLFilhos	List<RegistrosRetorno>	Será preenchido com uma lista do tipo [URLFilhos], onde cada item/URL representa uma coleção/filho do respectivo cadastro. Este link retornará um novo objeto [InformacoesRetorno] com todos os registros filhos correspondentes aos pais retornados na paginação.

Objeto [RegistrosRetorno]		
Campo	Tipo	Observação
Identificador	Alfanumérico	Será preenchido com o nome/identificador do cadastro ao qual os registros retornados pertencem.
IDEspaider	Inteiro	Será preenchido com o ID do registro no Espaider.
ListaCampos	List<CampoRetorno>	Lista do tipo “CampoRetorno” contendo todos os campos/informações do respectivo registro retornado.

Objeto [CampoRetorno]		
Campo	Tipo	Observação
Identificador	Alfanumérico	Será preenchido com o nome do campo ao qual a informação pertence.
Valor	- -	Possui o valor do campo retornado pelo Espaider. IMPORTANTE: O Tipo/formato do campo dependerá da informação de origem. Esta informação do Tipo estará especificada mais adiante no respectivo campo de retorno.

Objeto [URLFilhos]		
Campo	Tipo	Observação
Identificador	Alfanumérico	Será preenchido com o identificador do cadastro filho encontrado para a configuração informada.
URL	URL	Será preenchida com uma URL com acesso a busca das informações do cadastro filho vinculado à configuração informada. Esta URL retornará um novo retorno do tipo [InformacoesRetorno].

Abaixo um exemplo em C# realizando a leitura do retorno dos dados:

```
StringBuilder srt = new StringBuilder();
2 references
public void PercorreRetorno(InformacoesRetorno ObjetoRetorno)
{
    if (ObjetoRetorno.ListaRegistros != null)
    {
        //Percorrendo os registros retornados
        foreach (RegistrosRetorno registros in ObjetoRetorno.ListaRegistros)
        {
            //Imprimindo o identificador do cadastro retornado
            srt.AppendLine($"Identificador do cadastro: {registros.Identificador}");
            //ID ESPAIDER
            srt.AppendLine($"Identificador do IDESPAIDER: {registros.IDEspaider}");
            //Percorrendo a lista de membros do cadastro atual
            foreach (var campo in registros.ListaCampos)
            {
                //Imprimindo o identificador do campo
                srt.AppendLine($"Identificador Campo: {campo.Identificador}");
                if (campo.Identificador == "Documento")
                {
                    DownloadDocumento(campo.Valor);
                }
                //Imprimindo o valor retornado
                srt.AppendLine($"Valor: {campo.Valor}");
            }
            srt.AppendLine($"*****");
        }
    }
    if (ObjetoRetorno.ListaURLFilhos != null)
    {
        //Percorrendo a lista de cadastro filhos do registro atual (Deverá ser criado um método recursivo, poderá existir N registros filhos)
        foreach (var registroFilho in ObjetoRetorno.ListaURLFilhos)
        {
            //Imprimindo o identificador do cadastro retornado
            srt.AppendLine($"Identificador do cadastro: {registroFilho.Identificador}");

            //Imprimindo URL
            srt.AppendLine($"Identificador url: {registroFilho.URL}");

            //Busca informações das URLs retornadas
            BuscaInformacoesFilhos(registroFilho.URL);
        }
    }
    //Imprimindo a situação
    srt.AppendLine($"Situação: {ObjetoRetorno.Situacao}");
    //Imprimindo a mensagem de retorno
    srt.AppendLine($"Mensagem de retorno: {ObjetoRetorno.MensagemRetorno}");
    textBox1.Text = srt.ToString();
}
```

Neste exemplo foi criado um método “BuscaInformacoesFilhos” para percorrer a lista de URLs filhas e buscar as informações dos filhos vinculados à configuração.

Abaixo método em c#:

```
public void BuscaInformacoesFilhos(string url)
{
    //Endereço do sistema para acesso à API
    RestClient client = new RestClient("http://localhost/Esp5.01/WCF/WCFExportaDados/WCFExportaDados.svc");

    //API a ser acessar
    RestRequest request = new RestRequest(url, Method.GET);

    //Execução da chamada da API
    IRestResponse<InformacoesRetorno> retorno = client.Execute<InformacoesRetorno>(request);
    //Pegando o retorno já formatado
    var ObjetoRetorno = retorno.Data;

    //Percorrer o retorno
    PercorreRetorno(ObjetoRetorno);
}
```

Para à URL retornada deve-se realizar uma chamada utilizando o método Method.GET.

Perceber que no método definido acima o retorno gerado pela solicitação da URL é um objeto do tipo “InformacoesRetorno”, ou seja, um objeto com os mesmos atributos do método inicial (PercorrerRetorno).

Definição das classes utilizadas nos exemplos (em c#):

```
public class InformacoesRetorno
{
    2 references
    public List<RegistrosRetorno> ListaRegistros { get; set; }
    1 reference
    public char Situacao { get; set; }
    1 reference
    public string MensagemRetorno { get; set; }
    0 references
    public string URLPaginacao { get; set; }
    2 references
    public List<URLFilhos> ListaURLFilhos { get; set; }
}
2 references
public class RegistrosRetorno
{
    1 reference
    public string Identificador { get; set; }
    1 reference
    public long IDEspaider { get; set; }
    1 reference
    public List<CampoRetorno> ListaCampos { get; set; }
}
1 reference
public class CampoRetorno
{
    2 references
    public string Identificador { get; set; }
    2 references
    public string Valor { get; set; }
}
1 reference
public class URLFilhos
{
    1 reference
    public string Identificador { get; set; }
    2 references
    public string URL { get; set; }
}
```

9. Retorno (campos)

Segue abaixo especificação técnica dos campos configurados para serem retornados:

Identificador de origem: BI_SOLICITACOES_SUPORTEESPAIDER		
Campo/Identificador	Tipo	Observações
APROVADORATUAL	Alfanumérico	
ASSUNTOAREA	Alfanumérico	
CODIGO	Alfanumérico	
CRONOGRAMAATUAL	Alfanumérico	
DATAINICIOAPROVACAO	Data e hora	Data no formato dd/MM/yyyy HH:mm.
DATAMOVIMENTACAO	Data e hora	Data no formato dd/MM/yyyy HH:mm:ss.
ENCERRADOEM	Data e hora	Data no formato dd/MM/yyyy HH:mm.
NOME	Alfanumérico	
PASTACONSULTIVO	Alfanumérico	
PRAZOAPROVADOR	Data e hora	Data no formato dd/MM/yyyy.
PRAZOCRONOGRAMAATUAL	Data e hora	Data no formato .
PRAZOFINAL	Data e hora	Data no formato .
PRIORIDADE		Onde: "Alta" = "1"; "Normal" = "2"; "Baixa" = "3";.
RESPONSAVELPROJETO	Alfanumérico	
SITUACAOATUAL	Alfanumérico	
SOLUCAOAPLICADAEM		Onde:..
STATUSPROJETO		Onde:..
TIPOASSUNTO	Alfanumérico	
TIPOCHAMADO	Alfanumérico	
TRMESPAIDER	Alfanumérico	

Identificador de origem: BI_SOLICITACOES_SUPORTEESPAIDER_REQUISITOS		
Campo/Identificador	Tipo	Observações
DATACONCLUSAO	Data e hora	Data no formato dd/MM/yyyy.
DETALHAMENTOREQUISITO	Alfanumérico	
ENTREGA	Alfanumérico	
IDENTIFICADOR	Inteiro	
IDENTIFICADOR_ENTREGA	Inteiro	
IMPACTO		Onde:..
ORIGEMREQUISITO		Onde:..

PRIORIDADE		Onde:..
REQUISITO	Alfanumérico	
STATUSREQUISITO		Onde:..

Identificador de origem: BI_SOLICITACOES_SUPORTEESPAIDER_ENTREGAS		
Campo/Identificador	Tipo	Observações
DATACONCLUSAO	Data e hora	Data no formato dd/MM/yyyy.
DATAINICIO	Data e hora	Data no formato dd/MM/yyyy.
DETALHAMENTO	Alfanumérico	
ENTREGA	Alfanumérico	
IDENTIFICADOR	Inteiro	
ORDEM	Inteiro	
PRIORIDADE		Onde:..

Identificador de origem: BI_SOLICITACOES_SUPORTEESPAIDER_CRONOGRAMAS		
Campo/Identificador	Tipo	Observações
ATIVIDADE	Alfanumérico	
ATRASADO	Boleano	
CRIARAGENDACONSULTIVO	Boleano	
DATAALERTAPRAZO	Data e hora	Data no formato dd/MM/yyyy.
DATACONCLUSAO	Data e hora	Data no formato dd/MM/yyyy.
DATAINICIO	Data e hora	Data no formato dd/MM/yyyy.
DATANOVOPRAZO	Data e hora	Data no formato dd/MM/yyyy.
DATAPRAZO	Data e hora	Data no formato dd/MM/yyyy.
DETALHAMENTO	Alfanumérico	
FASEATIVIDADE		Onde:..
IDENTIFICADOR	Inteiro	
ITEM	Alfanumérico	
PRAZOCONFIRMADO		Onde:..
RESPONSAVEL	Alfanumérico	
SETORRESPONSAVEL	Alfanumérico	
STATUS	Alfanumérico	